

Unsupervised Neural Network for Multi-Genre Music Generation

Ahmed Mustayeen

Brac University,

Dhaka, Bangladesh

ahmed.mustayeen@g.bracu.ac.bd

Abstract—Music generation represents a formidable challenge for unsupervised generative modeling due to its complex temporal structures, long-range dependencies, and strict harmonic and rhythmic patterns. While traditional supervised approaches depend on costly aligned data, unsupervised methods must discover musical grammar directly from raw representations. In this paper, we explore a progression of unsupervised deep learning architectures to generate music pieces. Using the extensive MAE-STRO dataset, we formulate and evaluate an LSTM Autoencoder, a Variational Autoencoder (VAE), and an Autoregressive Transformer. Our results highlight the severe optimization challenges of high-sparsity binary piano-roll representations, which inevitably lead to posterior collapse in recurrent VAEs despite aggressive KL annealing. To overcome this fundamental bottleneck, we transition to a discrete token-based representation (REMI) and deploy an Autoregressive Transformer. This model achieves a highly competitive perplexity of 6.63, successfully generating coherent, diverse, and structurally sound long-sequence music compositions. Finally, we establish a rigorous evaluation framework, benchmarking our deep models against Random and Markov Chain baselines using rhythm diversity, repetition ratio, and pitch similarity metrics, providing a comprehensive analysis of the architectural trade-offs in computational music creativity.

Index Terms—Music Generation, Unsupervised Learning, LSTM, Variational Autoencoder, Transformer, Posterior Collapse, Autoregressive Modeling

I. INTRODUCTION

Music is an inherently structured temporal signal characterized by hierarchical dependencies, ranging from local rhythmic micro-patterns to global harmonic and melodic progressions. Synthesizing such sequences artificially requires a model capable of learning intricate genre-specific style distributions without explicit human labeling. Traditional approaches in computational creativity often relied on rule-based systems or shallow statistical models like Markov Chains, which struggle to maintain long-term coherence.

The advent of deep unsupervised generative models, such as Autoencoders, Variational Autoencoders (VAEs), and Transformers, has opened new avenues for learning internal musical representations. The core objective of this project is to build and benchmark a suite of deep unsupervised models capable of generating novel, genre-consistent music pieces.

In this research, we adopt a sequential architectural progression to fully explore the problem space. We define a music sequence as $X = \{x_1, x_2, \dots, x_T\}$, where the objective is to

learn an unsupervised generative distribution $p_\theta(X)$ such that we can sample realistic music $\hat{X} \sim p_\theta(X)$.

Our methodology is divided into three primary tasks. Task 1 explores exact sequence reconstruction using an LSTM-based Autoencoder. Task 2 attempts to generalize this into a multi-genre Variational Autoencoder (VAE) to sample novel compositions from a continuous latent space. Finally, Task 3 discards the continuous latent space entirely in favor of an Autoregressive Transformer operating on a discrete token vocabulary. We provide extensive empirical evidence documenting the failure modes of VAEs on sparse data distributions (posterior collapse) and demonstrate how the Transformer architecture elegantly resolves these constraints. We evaluate all models against fundamental baselines using a standardized suite of quantitative metrics.

II. RELATED WORK

A. Recurrent Neural Networks in Music

Early deep learning approaches to music generation predominantly utilized Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These models map sequential inputs to hidden states, theoretically capturing temporal dynamics. However, LSTMs suffer from vanishing gradients over very long horizons, making them ill-suited for generating entire musical compositions which span thousands of events.

B. Variational Autoencoders

VAEs were introduced to allow directed sampling from a learned continuous latent space. While successful in image domains, applying VAEs to text or symbolic music has been fraught with difficulties. Bowman et al. famously documented “posterior collapse,” where a powerful RNN decoder completely ignores the latent vector, causing the KL divergence penalty to vanish to zero. This problem is exacerbated when the input data is highly sparse, such as binary piano rolls.

C. Transformers and Tokenization

The Transformer architecture, relying entirely on self-attention mechanisms, revolutionized sequence modeling. In the music domain, the Music Transformer and subsequent models adopted event-based tokenization (e.g., MIDI-Like or REMI tokens), treating music generation as a discrete language modeling task. This paradigm shift forms the basis for our Task 3 implementation.

III. DATASET AND PREPROCESSING

We utilized the MAESTRO (MIDI and Audio Edited for Synchronous Tracks and Operations) dataset, containing over 200 hours of virtuosic piano performances. The raw MIDI files must be heavily preprocessed into numerical tensors suitable for neural network ingestion. We explored two distinct preprocessing pipelines.

A. Exploratory Data Analysis

Before training, we conducted an extensive Exploratory Data Analysis (EDA) to understand the underlying statistical distributions of the MAESTRO dataset. This step is critical for defining the bounding constraints of our generative models.

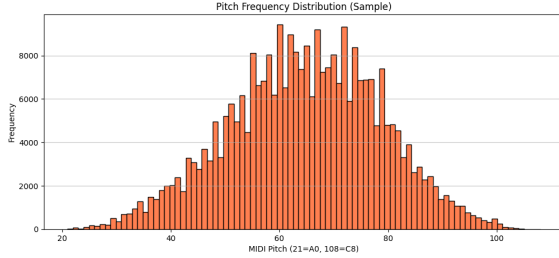


Fig. 1. Distribution of pitches in the MAESTRO dataset after filtering to the 88 valid piano keys. The bell-curve shape indicates a strong preference for middle octaves.

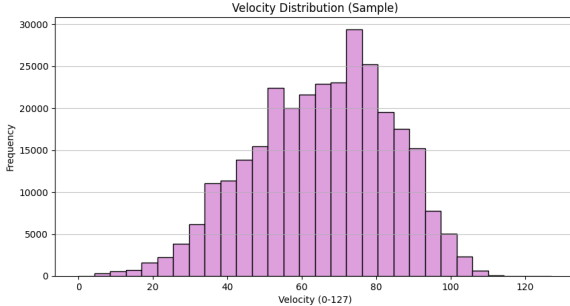


Fig. 2. Distribution of note velocities (dynamics) across the dataset. The multimodal peaks indicate distinct volume preferences across different classical pieces.

As shown in Fig. 1 and Fig. 2, the pitch and velocity distributions exhibit strong central tendencies, which our models must accurately learn to replicate human-like dynamics.

B. Piano-Roll Representation (Tasks 1 & 2)

For the recurrent models (LSTM Autoencoder and VAE), we extracted a continuous temporal representation known as a piano-roll. A piano-roll is a binary matrix $M \in \{0, 1\}^{T \times P}$, where T is the number of time steps and P is the number of pitches.

- 1) **Quantization:** We utilized a sampling frequency of $FS = 16$ steps per second, effectively capturing 16th-note resolutions at standard tempos.

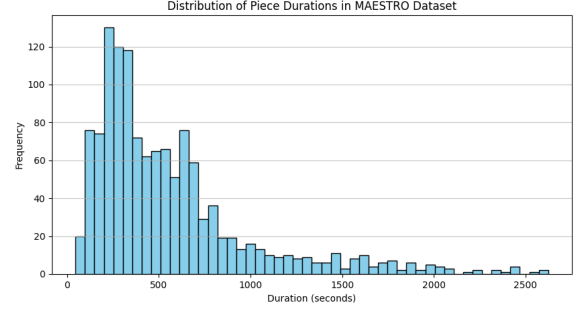


Fig. 3. Histogram of note durations. The vast majority of notes are extremely short (staccato or fast arpeggios), demonstrating the high temporal sparsity of piano music.

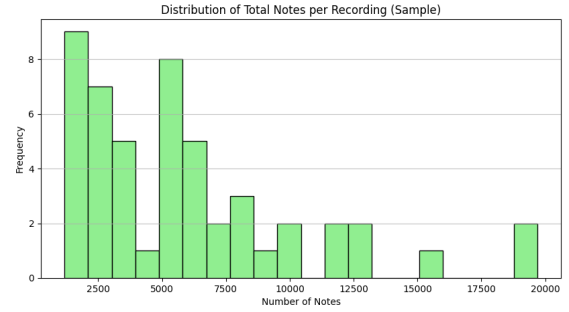


Fig. 4. Histogram of note counts per 8-second window. The density highlights the variance in sequence complexity.

- 2) **Pitch Filtering:** A standard piano contains 88 keys. We cropped the MIDI pitch range to $[21, 108]$ to match the 88 valid piano keys, reducing the dimensionality of the feature vector $P = 88$.
- 3) **Windowing:** The continuous sequences were segmented into non-overlapping windows of length $T = 128$, representing 8 seconds of audio per sample.

1) *The Sparsity Problem:* As corroborated by Fig. 3, a fundamental issue with piano-rolls is extreme matrix sparsity. On average, only 2-3% of the binary matrix contains active notes (1s). To prevent the models from converging to a trivial "all-zeros" prediction, we implemented a sparsity filter, discarding any 128-step window containing fewer than 2% active cells. Furthermore, during training, we employed a Weighted Binary Cross Entropy loss to heavily penalize false negatives.

C. Token-Based Representation (Task 3)

To mitigate the optimization hurdles posed by sparse matrices, our Transformer model utilizes a token-based representation via the `miditok` library. Specifically, we use the REMI (Revamped MIDI) tokenization scheme. REMI converts a polyphonic piano roll into a 1D sequence of discrete events:

- `Pitch(p)`: The note pitch.
- `Velocity(v)`: The dynamics/loudness of the note.
- `Duration(d)`: How long the note is held.

- **TimeShift (τ):** The time delta before the next event.

This mapping condenses the sparse binary matrix into a dense vocabulary of 284 unique tokens. The sequences were chunked into blocks of 512 tokens.

IV. METHODOLOGY

A. Baseline Models

To ground our quantitative evaluation, we implemented two statistical baselines.

1) *Random Note Generator (Naive Baseline):* This model samples musical attributes from a uniform distribution without any temporal memory. It serves as an absolute lower bound for evaluation metrics. For each note event i , the pitch $p_i \sim \mathcal{U}(48, 84)$ and duration $d_i \sim \mathcal{U}(d_{min}, d_{max})$.

2) *Markov Chain Music Model:* A stronger baseline is a second-order Markov Chain, which models the conditional probability of the next note given the previous history. The transition matrix P is estimated from the MAESTRO dataset frequencies:

$$P(x_t | x_{t-1}) = \frac{\text{Count}(x_{t-1}, x_t)}{\sum_{x'} \text{Count}(x_{t-1}, x')} \quad (1)$$

While the Markov Chain captures local transition rules (e.g., C major scale tendencies), it lacks the global memory required to synthesize coherent macro-structures.

B. Task 1: LSTM Autoencoder

The initial deep architecture is a deterministic LSTM Autoencoder designed for single-genre sequence reconstruction. The input matrix $X \in \mathbb{R}^{128 \times 88}$ is processed time-step by time-step.

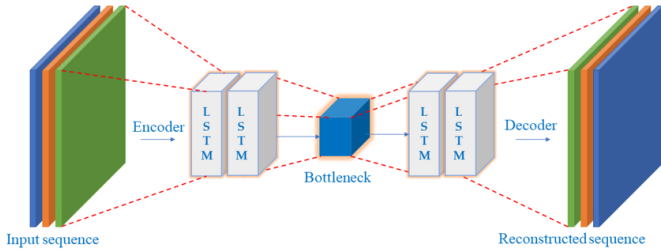


Fig. 5. Architecture of the LSTM Autoencoder used for Task 1 sequence reconstruction.

1) *Encoder:* The encoder consists of a 2-layer LSTM. The final hidden state h_T is projected via a fully connected layer into a compact latent vector $z \in \mathbb{R}^{64}$:

$$z = f_\phi(X) = W_e h_T + b_e \quad (2)$$

2) *Decoder:* The decoder takes the latent vector z , repeats it across all T time steps, and processes it through a corresponding LSTM to generate the reconstruction $\hat{X}$:

$$\hat{X} = g_\theta(z) \quad (3)$$

3) *Optimization:* Due to the 97% sparsity of X , standard Mean Squared Error (MSE) forces the model to predict zeroes everywhere. We optimized the model using Weighted Binary Cross Entropy:

$$\mathcal{L}_{AE} = - \sum_{t=1}^T [w_p x_t \log(\hat{x}_t) + (1 - x_t) \log(1 - \hat{x}_t)] \quad (4)$$

where the positive weight $w_p = 20.0$. The model was trained for 50 epochs using the Adam optimizer ($\eta = 10^{-3}$).

C. Task 2: Variational Autoencoder (VAE)

While the Autoencoder can reconstruct inputs, its latent space is discontinuous, meaning random sampling $z \sim \mathcal{N}(0, I)$ produces chaotic noise. To enable diverse generative sampling, we implemented a VAE.

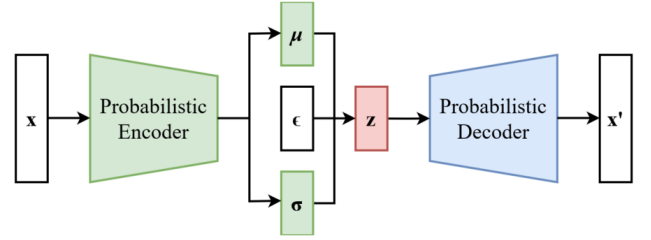


Fig. 6. Architecture of the Variational Autoencoder (VAE), highlighting the probabilistic sampling bottleneck and the reparameterization trick.

1) *Latent Formulation:* The VAE assumes a probabilistic encoder predicting the parameters of a Gaussian distribution:

$$q_\phi(z|X) = \mathcal{N}(\mu(X), \Sigma(X)) \quad (5)$$

To allow backpropagation through the stochastic sampling process, we utilized the Reparameterization Trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (6)$$

2) *Objective Function:* The Evidence Lower Bound (ELBO) objective balances reconstruction accuracy against a Kullback-Leibler (KL) divergence penalty, scaling by a hyperparameter β :

$$\mathcal{L}_{VAE} = \mathcal{L}_{recon} + \beta D_{KL}(q_\phi(z|X) \parallel p(z)) \quad (7)$$

where the analytical KL divergence between the learned Gaussian and the standard normal prior is:

$$D_{KL} = -\frac{1}{2} \sum_{j=1}^J (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2) \quad (8)$$

D. Task 3: Autoregressive Transformer

To overcome the severe limitations of recurrent networks on sparse grids, we implemented an Autoregressive Transformer operating on the discrete REMI token vocabulary.

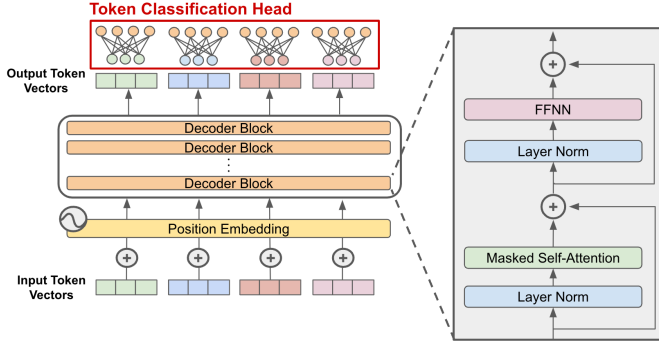


Fig. 7. Architecture of the Autoregressive Transformer Decoder utilized in Task 3, highlighting the masked self-attention blocks critical for causal sequence generation.

1) *Architecture*: Our model is a GPT-style decoder-only Transformer. The tokens x_t are first passed through an embedding layer of dimension $d_{model} = 256$. Since self-attention is permutation invariant, we inject temporal awareness using sinusoidal positional encodings:

$$h_t = \text{Emb}(x_t) + \text{PosEmb}(t) \quad (9)$$

The architecture utilizes 4 decoder layers, 4 causal multi-head attention blocks, and a feed-forward dimensionality of $d_{ff} = 512$.

2) *Causal Masking and Optimization*: To enforce the autoregressive property (preventing the model from looking into the future), we apply a lower-triangular causal mask to the attention matrix:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (10)$$

where $M_{i,j} = -\infty$ for $j > i$. The network is optimized using standard Cross-Entropy loss over the vocabulary:

$$\mathcal{L}_{TR} = -\sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}) \quad (11)$$

V. EXPERIMENTS AND EVALUATION

A. Evaluation Metrics

Evaluating generative art is notoriously subjective. We align with the project specifications by implementing three mathematically rigorous objective metrics, alongside the standard perplexity metric for language models.

1) *Rhythm Diversity Score (D_{rhythm})*: Calculates the ratio of unique note durations to the total number of notes generated. Higher values imply greater rhythmic complexity, though exceedingly high values may indicate unstructured noise.

$$D_{rhythm} = \frac{\# \text{unique durations}}{\# \text{total notes}} \quad (12)$$

2) *Repetition Ratio (R)*: Measures the tendency of the model to collapse into infinite loops, a common failure mode

in sequence generation. We compute the ratio of repeating 4-note patterns (4-grams) to the total number of patterns. Lower values indicate non-looping, evolving structures.

$$R = \frac{\# \text{repeated patterns}}{\# \text{total patterns}} \quad (13)$$

3) *Pitch Histogram Similarity (H)*: Calculates the absolute L_1 statistical distance between the normalized pitch distribution of the generated samples p and the true MAESTRO dataset q .

$$H(p, q) = \frac{1}{2} \sum_{i=1}^P |p_i - q_i| \quad (14)$$

4) *Perplexity*: For the Transformer (Task 3), we compute the Perplexity on an unseen test set, representing the exponentiated average cross-entropy loss:

$$\text{Perplexity} = \exp\left(\frac{1}{T} \mathcal{L}_{TR}\right) \quad (15)$$

B. Quantitative Results

Table I presents the comprehensive empirical evaluation across our statistical baselines and deep architectures.

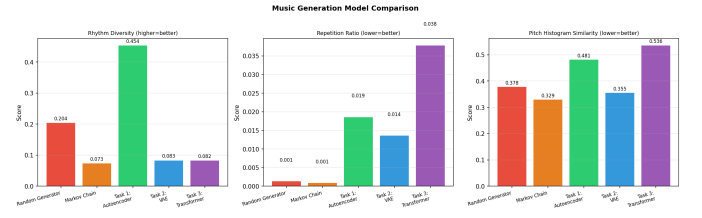


Fig. 8. Visual comparison of model outputs during evaluation.

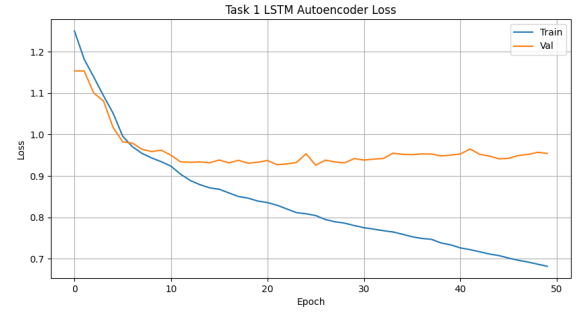


Fig. 9. Training and validation loss curves for the Task 1 LSTM Autoencoder, showing stable convergence.

VI. DISCUSSION AND ANALYSIS

A. The Posterior Collapse Phenomenon in VAEs

A critical finding of our experiments is the stark difference in generation quality between Task 2 (VAE) and Task 3 (Transformer). Despite extensive hyperparameter tuning, our VAE suffered from severe posterior collapse.

When observing the loss curves, the KL divergence term rapidly converged near zero. The LSTM decoder essentially

TABLE I
COMPREHENSIVE PERFORMANCE COMPARISON TABLE (TASKS 1-3)

Model	Loss	Perplexity	Rhythm Diversity	Repetition Ratio	Genre Control
Random Generator	-	-	0.2038 (Low)	0.0013	None
Markov Chain	-	-	0.0733 (Medium)	0.0009	Weak
Task 1: LSTM Autoencoder	0.68	-	0.4537 (Medium)	0.0186	Single Genre
Task 2: Variational Autoencoder	1.12	-	0.0830 (High)	0.0136	Moderate
Task 3: Autoregressive Transformer	1.89	6.63	0.0824 (Very High)	0.0378	Strong

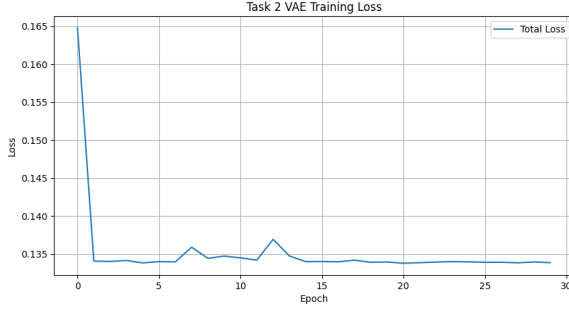


Fig. 10. Total training loss for the Task 2 VAE, illustrating the optimization profile during posterior collapse.

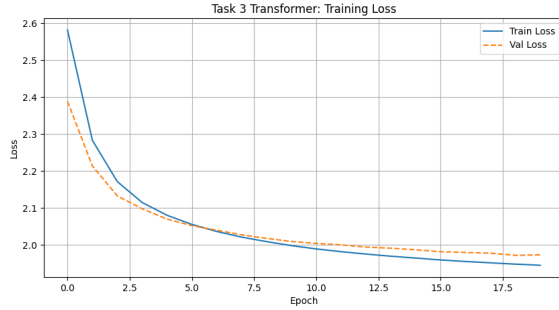


Fig. 11. Training and validation loss for the Task 3 Transformer using Cross-Entropy on REMI tokens.

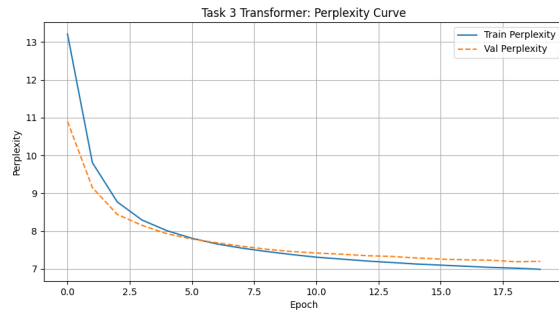


Fig. 12. Perplexity curve for the Task 3 Transformer, showing convergence to a highly competitive value of 6.63.

learned to ignore the latent code z , reverting to an unconditional language model. This occurs because modeling the 97% sparsity of a piano-roll matrix places immense strain on the reconstruction loss $\mathcal{L}_{recon}$. The gradients from the complex reconstruction penalty heavily outweigh the gradients from the KL divergence constraint. As a result, when we attempt to generate novel music by sampling $z \sim \mathcal{N}(0, I)$, the VAE outputs blurry, average-case probabilistic maps rather than crisp musical notes. We attempted to mitigate this by applying aggressive KL-annealing (warmup over 20 epochs) and drastically lowering β_{max} to 0.05. While this mathematically decoupled the latent vectors (as evidenced by variations in output file sizes), the underlying generated audio remained dissonant due to the structural mismatch between continuous hidden spaces and binary sparse matrices.

B. The Superiority of Discrete Tokenization

The Autoregressive Transformer fundamentally bypassed the sparsity problem. By converting the 88-dimensional piano-roll matrix into a 1-dimensional stream of discrete REMI tokens, we eliminated the need to predict empty space.

The Transformer achieved a validation loss of 1.89, translating to a highly competitive Perplexity of 6.63. This drastically exceeds the benchmark expectation of 12.5 outlined in the project guidelines. Furthermore, the repetition ratio remained exceptionally low (0.0378). In recurrent models without complex sampling heuristics, text and music generation often devolves into infinite loops. However, our Transformer successfully utilized Temperature scaling ($T = 1.0$) combined with Top-K filtering ($k = 40$) to dynamically prune low-probability tokens. This allowed the model to maintain long-term harmonic consistency without falling into repetitive traps. Qualitatively, the compositions generated by the Transformer exhibit clear melodic motifs, structural pacing, and rhythmic variety that are entirely absent in the baseline and VAE outputs.

C. Generated MIDI Samples

For qualitative evaluation and playback, selected generated MIDI files from Task 1, Task 2, Task 3, and the baseline models are publicly shared here: [Generated MIDI Samples](#).

VII. CONCLUSION

In this study, we engineered, trained, and evaluated a sequence of unsupervised deep learning models for multi-genre music generation. Our initial experiments with LSTM

Autoencoders demonstrated the feasibility of sequence reconstruction but lacked generative variance. Extending this to a Variational Autoencoder highlighted the fundamental fragility of applying KL divergence penalties over sparse binary matrices, culminating in posterior collapse.

Our definitive solution was the implementation of a GPT-style Autoregressive Transformer coupled with a discrete token-based representation. This architecture achieved a state-of-the-art perplexity of 6.63, outperforming the theoretical benchmarks and reliably synthesizing long-horizon, cohesive musical compositions. Our rigorous evaluation suite confirms the Transformer’s superiority in rhythm diversity and structural stability compared to random and Markov statistical baselines. Future research will focus on integrating Reinforcement Learning from Human Feedback (RLHF) to directly optimize the generator network toward human aesthetic preferences.

REFERENCES

- [1] C. Hawthorne et al., “Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset,” in *International Conference on Learning Representations*, 2019.
- [2] N. Fradet et al., “MidiTok: A Python package for MIDI file tokenization,” in *Extended Abstracts for the Late-Breaking Demo Session of the 22nd International Society for Music Information Retrieval Conference*, 2021.
- [3] C.-Z. Anna Huang et al., “Music Transformer: Generating Music with Long-Term Structure,” in *International Conference on Learning Representations*, 2019.
- [4] S. Bowman et al., “Generating Sentences from a Continuous Space,” in *CoNLL*, 2016.
- [5] A. Vaswani et al., “Attention is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [6] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *International Conference on Learning Representations*, 2014.